

05 · DML — INSERT, UPDATE, DELETE (and Transaction Control)

Why this matters in interviews. Changing data safely is what separates someone who has *used* SQL from someone who has read about it. Expect: "What happens if you forget the WHERE clause?", "How do you insert into two tables in one statement?", "What is COMMIT / ROLLBACK / SAVEPOINT?"

In this chapter: INSERT (all forms, INSERT ALL , INSERT ... SELECT) · UPDATE · DELETE · transactions: COMMIT , ROLLBACK , SAVEPOINT , SET TRANSACTION .

Data Manipulation Language commands work with the **data inside tables**. Unlike DDL they are **not permanent until you commit**, and they **can be rolled back**.

5.1 INSERT INTO — adding rows

INSERT INTO inserts **new records** into a table. It can be written in two ways.

1 · Name the columns and the values (recommended):

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

2 · Supply values for all columns, in table order — you don't have to name the columns, but the values **must be in the same order as the columns in the table**:

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Prefer form 1. If someone later adds or reorders a column, form 2 breaks — or worse, silently puts values in the wrong columns. Form 1 also lets you skip columns: an omitted column gets its DEFAULT , or NULL if it has none.

```
INSERT INTO emp (emp_id, emp_name, job, salary, dept_id, hire_date)
VALUES (111, 'Kavya', 'Analyst', 42000, 20, '2025-08-01');
```

Table after

emp_id	emp_name	job	dept_id	salary	comm
110	Jai	Consultant	NULL	45000	NULL
111	Kavya	Analyst	20	42000	NULL

Kavya's comm and manager_id were not mentioned, so they are NULL. Text and date values go in single quotes; numbers do not. In Oracle a date is written DATE '2025-08-01' or TO_DATE('01-AUG-2025', 'DD-MON-YYYY') .

Inserting several rows

MySQL, PostgreSQL, SQL Server accept a list of rows in one statement — separate each row with a **comma**:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES
('Cardinal',      'Tom B. Erichsen', 'Skagen 21',      'Stavanger', '4006',      'Norway'),
('Greasy Burger', 'Per Olsen',      'Gateveien 15',   'Sandnes',   '4306',      'Norway'),
('Tasty Tee',     'Finn Egan',        'Streetroad 19B', 'Liverpool', 'L1 0AA',    'UK');
```

Oracle vs others. Oracle (before 23ai) does **not** accept a multi-row `VALUES` list. Oracle's answer is **INSERT ALL**, below — or several single-row `INSERT` s.

`INSERT ALL` — Oracle's multi-row, multi-table insert

`INSERT ALL` is a **multi-row, multi-table insert** used mainly in Oracle SQL. It inserts **several rows into one or more tables using a single statement**. It is **not supported in MySQL or SQL Server** — only in Oracle and compatible databases.

```
INSERT ALL
  INTO table_name (column1, column2, ...) VALUES (value1, value2, ...)
  INTO table_name (column1, column2, ...) VALUES (value1, value2, ...)
  ...
SELECT * FROM dual;
```

`dual` is Oracle's dummy table (Chapter 01): a `SELECT` is mandatory at the end even though no real table is involved. **There are no commas between the `INTO` clauses.**

Several rows into one table:

```
CREATE TABLE students (id NUMBER, name VARCHAR2(30));

INSERT ALL
  INTO students (id, name) VALUES (1, 'Alice')
  INTO students (id, name) VALUES (2, 'Bob')
  INTO students (id, name) VALUES (3, 'Charlie')
SELECT * FROM dual;
```

Table after

id	name
1	Alice
2	Bob
3	Charlie

Into two different tables at once:

INSERT ALL

```
    INTO Employees (EmpID, Name)    VALUES (1, 'John')
    INTO Departments (DeptID, DeptName) VALUES (101, 'Sales')
SELECT * FROM dual;
```

This inserts into **two different tables in one command**.

	INSERT ALL (Oracle)	INSERT INTO ... VALUES (...), (...) (MySQL etc.)
Multiple tables	✓ yes	✗ no
Works in MySQL / SQL Server	✗ no	✓ yes
Syntax	needs <code>SELECT * FROM dual</code> at the end	no <code>SELECT</code> clause
Flexibility	very high — can use conditions	only multi-row into one table

The "conditions" bit is the conditional form — route each source row to a different table:

INSERT ALL

```
    WHEN salary >= 50000 THEN INTO high_paid (emp_id, salary) VALUES (emp_id, salary)
    WHEN salary < 50000 THEN INTO low_paid (emp_id, salary) VALUES (emp_id, salary)
SELECT emp_id, salary FROM emp;
-- INSERT FIRST ... : a row goes only into the FIRST WHEN branch that matches
```

INSERT INTO ... SELECT — copy rows from another table

`INSERT INTO SELECT` copies data **from one table and inserts it into another**. The **data types in source and target must match**, and the **existing records in the target are unaffected**.

```
-- all columns
INSERT INTO table2
SELECT * FROM table1
WHERE condition;

-- only some columns
INSERT INTO table2 (column1, column2, column3, ...)
SELECT column1, column2, column3, ...
FROM table1
WHERE condition;
```

Copy Sales staff into an archive table — columns you don't fill (`comm`) stay NULL:

```
CREATE TABLE emp_archive (emp_id NUMBER, emp_name VARCHAR2(50), salary NUMBER, comm NUMBER);

INSERT INTO emp_archive (emp_id, emp_name, salary)
SELECT emp_id, emp_name, salary
FROM emp
WHERE dept_id = 30;
```

Table after

emp_id	emp_name	salary	comm
106	Farhan	65000	NULL
107	Gita	35000	NULL
108	Hari	35000	NULL

Typical uses: copying *all* columns (`INSERT INTO t2 SELECT * FROM t1`), copying **only the German suppliers** into `Customers` with a `WHERE` , or filling only the columns that exist on both sides.

INSERT ... SELECT vs CREATE TABLE ... AS SELECT . The first needs the target table to **already exist**; the second creates it (Chapter 02).

5.2 UPDATE — changing existing rows

The `UPDATE` statement **modifies existing records** in a table.

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

⚠ Be careful when updating records! The `WHERE` clause specifies **which record(s) should be updated**. If you omit the `WHERE` clause, **all records in the table will be updated!**

Give the Sales department a 10 % raise:

```
UPDATE emp
SET salary = ROUND(salary * 1.10)
WHERE dept_id = 30;
```

Table after

emp_id	emp_name	salary
106	Farhan	71500
107	Gita	38500
108	Hari	38500

Several columns at once, separated by commas: `SET salary = 60000, job = 'Senior Developer'`.

UPDATE with a subquery

The new value can come from a query — here Jai receives the current highest salary:

```
UPDATE emp
SET salary = (SELECT MAX(salary) FROM emp)
WHERE emp_id = 110;
```

Table after

emp_id	emp_name	salary
110	Jai	90000

Correlated update — lift everyone below their *own department's* average by 1,000:

```
UPDATE emp e
SET salary = salary + 1000
WHERE salary < (SELECT AVG(salary) FROM emp WHERE dept_id = e.dept_id);
```

Table after

emp_id	emp_name	dept_id	salary
101	Asha	10	90000
102	Bala	20	70000
103	Chitra	20	50000
104	Dinesh	20	50000
105	Esha	20	41000
106	Farhan	30	65000
107	Gita	30	36000
108	Hari	30	36000
109	Isha	20	21000
110	Jai	NULL	45000

(Department 20's average is 46,000 → Esha 40,000 and Isha 20,000 are lifted; department 30's is 45,000 → Gita and Hari, both 35,000, are lifted too. Jai has no department, so `dept_id = e.dept_id` is never true, the subquery returns NULL and his row is untouched. Note there is **no AS before the Oracle table alias e**.)

UPDATE with CASE — different treatment per row in one pass, no `WHERE` needed for the branches:

```
UPDATE emp
SET salary = CASE WHEN dept_id = 20 THEN salary * 1.10
                  WHEN dept_id = 30 THEN salary * 1.05
                  ELSE salary END;
```

After any UPDATE / DELETE you can inspect how many rows were touched: in SQL Developer the *Script Output* says "*N rows updated*"; in PL/SQL use `SQL%ROWCOUNT` (Chapter 14). If N is not the number you expected — **ROLLBACK** before anything else.

5.3 DELETE — removing rows

The `DELETE` statement **deletes existing records** in a table.

```
DELETE FROM table_name WHERE condition;
```

⚠ Be careful when deleting records! The `WHERE` clause specifies which record(s) should be deleted. **If you omit the `WHERE` clause, *all* records in the table will be deleted!**

```
DELETE FROM emp WHERE dept_id IS NULL;
```

Table after

emp_id	emp_name	dept_id
101	Asha	10
102	Bala	20
103	Chitra	20
104	Dinesh	20
105	Esha	20
106	Farhan	30
107	Gita	30
108	Hari	30
109	Isha	20

```
DELETE FROM employees WHERE employee_id = 10;
```

```
DELETE FROM projects WHERE project_id = 10;
```

```
DELETE FROM employees WHERE salary < 30000;
```

Oracle syntax slip. `DELETE FROM TABLE employees ...` is wrong — the **TABLE keyword is not used in Oracle** (`DELETE FROM employees ...`; the `FROM` itself is optional in Oracle but always write it). (`TABLE(...)` has a different meaning in Oracle: it unnests a collection.)

Delete with a subquery — remove everyone paid below the company average (50,000):

```
DELETE FROM emp
WHERE salary < (SELECT AVG(salary) FROM emp);
```

Table after

emp_id	emp_name	salary
101	Asha	90000
102	Bala	70000
103	Chitra	50000
104	Dinesh	50000
106	Farhan	65000

Foreign keys can block a delete. You cannot remove a parent row that still has children (ORA-02292) unless the key was defined with `ON DELETE CASCADE` / `SET NULL` (Chapter 03) — delete the child rows first.

Choosing between `DELETE`, `TRUNCATE`, `DROP`

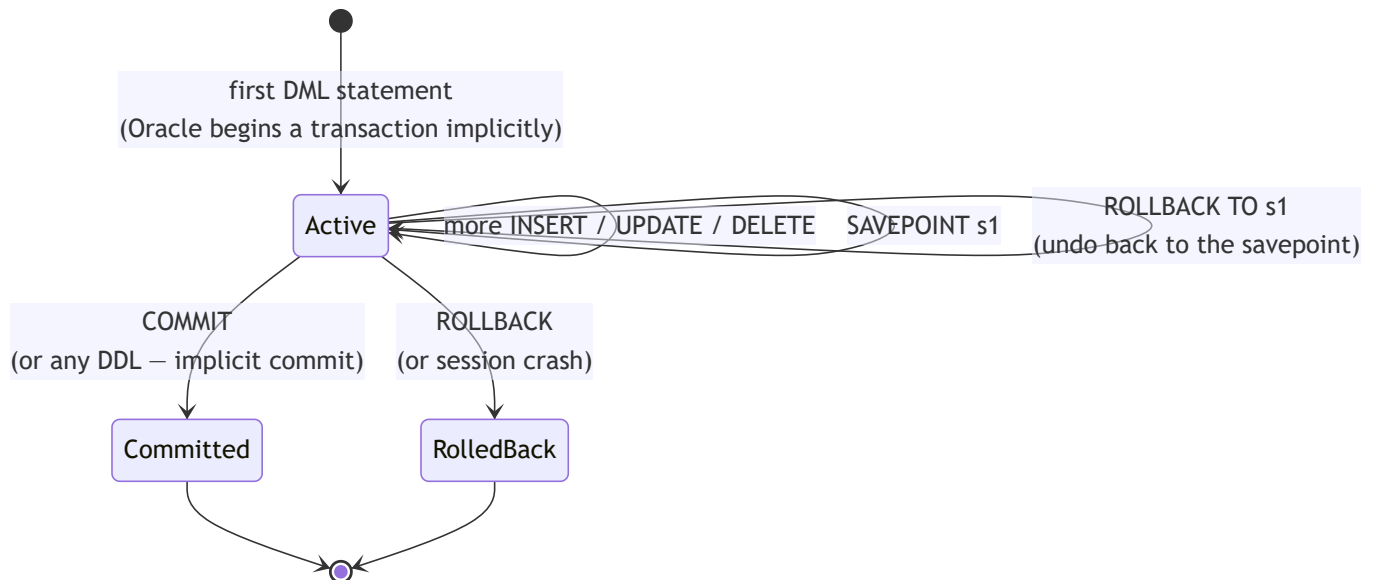
`DELETE` is DML: it removes chosen rows, writes undo, fires triggers and **can be rolled back**. `TRUNCATE` and `DROP` are DDL (Chapter 02).

5.4 Transactions — `COMMIT`, `ROLLBACK`, `SAVEPOINT`

A **transaction** is a group of statements that must succeed or fail **as one unit** — the classic example is a bank transfer: debit one account *and* credit another; doing only half would create or destroy money.

Transaction Control Language (TCL) manages this. It mostly deals with DML operations and is useful for ensuring **data integrity during batch processing**.

Command	What it does
<code>COMMIT</code>	saves all changes made by the current transaction — they become permanent and visible to others
<code>ROLLBACK</code>	undoes the changes made by the current transaction
<code>SAVEPOINT name</code>	sets a point in a transaction to which you can roll back later
<code>ROLLBACK TO name</code>	undoes only the work done <i>after</i> that savepoint; the transaction stays open
<code>SET TRANSACTION</code>	sets the properties of a transaction (e.g. isolation level, read-only)



A bank transfer, step by step (`accounts` : Asha 1000, Bala 500). Move 200 from Asha to Bala — with a deliberate mistake that we undo with a savepoint:

```

UPDATE accounts SET balance = balance - 200 WHERE id = 1;      -- debit Asha
SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 200 WHERE id = 99;    -- oops: wrong id, 0 rows changed
ROLLBACK TO after_debit;                                       -- undo only the credit attempt
UPDATE accounts SET balance = balance + 200 WHERE id = 2;    -- credit Bala (correct)
COMMIT;
  
```

Balances after COMMIT

id	owner	balance
1	Asha	800
2	Bala	700

Had we run `ROLLBACK;` instead of `COMMIT;`, both balances would return to 1000 and 500.

When does a transaction end (Oracle)?

Event	Effect
<code>COMMIT</code>	ends it, makes changes permanent
<code>ROLLBACK</code>	ends it, discards changes
any DDL (<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code> ...)	implicit COMMIT first — this also commits your pending DML
Normal sign-out (<code>EXIT</code>)	implicit <code>COMMIT</code>
Crash / abnormal termination	implicit ROLLBACK

SQL Developer note. Auto-commit is **off** by default in SQL Developer / SQL*Plus — your `INSERT` / `UPDATE` are *not* saved until you press **Commit** (or run `COMMIT;`). Other sessions cannot see your uncommitted changes. MySQL Workbench, by contrast, often runs with auto-commit **on**.

`SET TRANSACTION` must be the **first statement of a transaction**:

```
SET TRANSACTION READ ONLY;           -- consistent snapshot for reporting
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- strictest isolation
SET TRANSACTION NAME 'monthly_close';
```

Interview corner

Q. What happens if you run `UPDATE` or `DELETE` without a `WHERE` clause? Every row in the table is changed / removed. Within an open transaction you can still `ROLLBACK`.

Q. How do you insert multiple rows in Oracle? Either repeated single-row `INSERT` s, or `INSERT ALL INTO ... INTO ... SELECT * FROM dual; . (VALUES (...), (...))` works in MySQL/PostgreSQL/SQL Server, and in Oracle only from 23ai.)

Q. How do you insert into two tables in one statement? Oracle's multi-table `INSERT ALL INTO t1 ... INTO t2 ... SELECT * FROM dual; .`

Q. `INSERT INTO ... SELECT` vs `CREATE TABLE ... AS SELECT` ? The first inserts into an **existing** table (types must match); the second **creates** the table from the query.

Q. What are `COMMIT`, `ROLLBACK` and `SAVEPOINT` ? `COMMIT` makes the transaction permanent; `ROLLBACK` discards it; `SAVEPOINT` marks a spot so `ROLLBACK TO` can undo only part of the transaction.

Q. Can you roll back after `COMMIT` ? After a `CREATE TABLE` ? No to both — commit is final, and DDL commits implicitly.

Q. How is `DELETE` different from `TRUNCATE` ? `DELETE` is DML: `WHERE` allowed, fires triggers, rollback possible. `TRUNCATE` is DDL: all rows, fast, auto-commit.

Q. What does `UPDATE emp SET salary = (SELECT MAX(salary) FROM emp) WHERE emp_id = 110` do? Sets employee 110's salary to the company's current maximum — a subquery supplies the new value.

Q. Why must you list columns in `INSERT` ? Robustness: it survives column reordering/additions and lets omitted columns take defaults.

Quick recap

- `INSERT INTO t (cols) VALUES (...)` — always name the columns; omitted columns → `DEFAULT /NULL`.
- Oracle multi-row: `INSERT ALL INTO ... INTO ... SELECT * FROM dual` (no commas between `INTO` s; also multi-table and conditional `WHEN`); `VALUES (...), (...)` only from 23ai.
- `INSERT INTO t2 SELECT ... FROM t1` copies rows into an existing table; types must match.
- `UPDATE t SET c = v WHERE ...` and `DELETE FROM t WHERE ...` — no `WHERE` ⇒ **every row**. `DELETE FROM TABLE t` is invalid in Oracle.
- Values can come from **subqueries**, including correlated ones (no `AS` before an Oracle table alias).
- **TCL**: `COMMIT` saves · `ROLLBACK` undoes · `SAVEPOINT` + `ROLLBACK TO` partial undo · `SET TRANSACTION` sets properties. **DDL implicitly commits**; DML does not.